

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

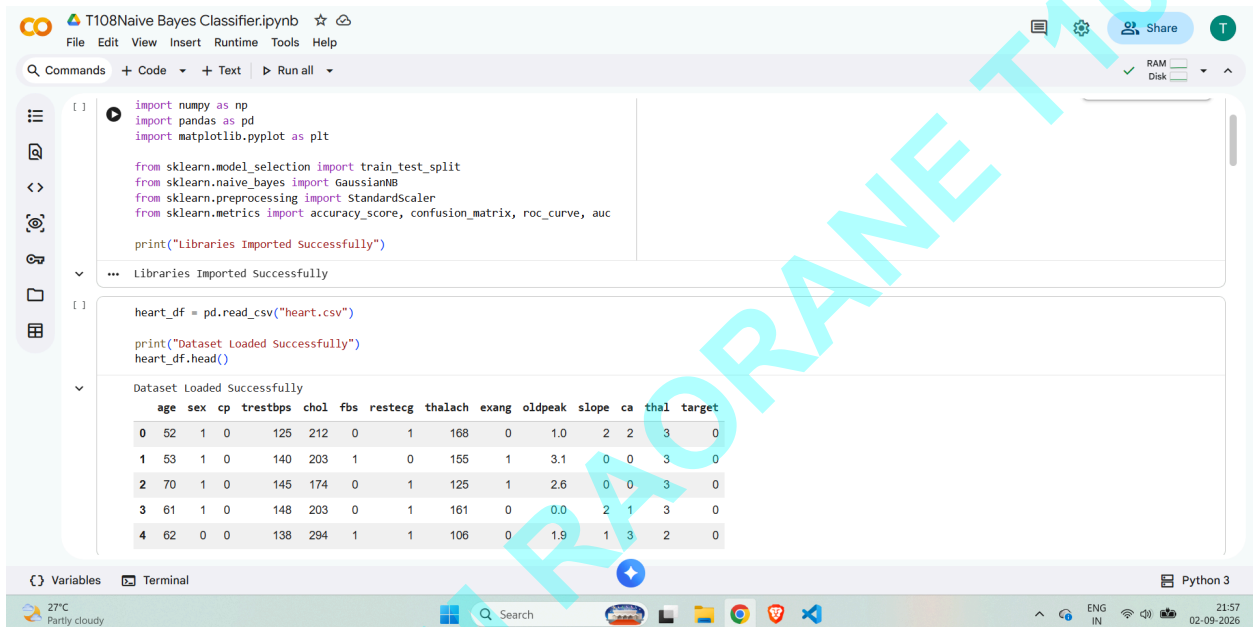
AIM : Naive Bayes' Classifier

Implement the Naive Bayes' algorithm for classification.

Train a Naive Bayes' model using a given dataset and calculate class probabilities.

Evaluate the accuracy of the model on test data and analyze the results.

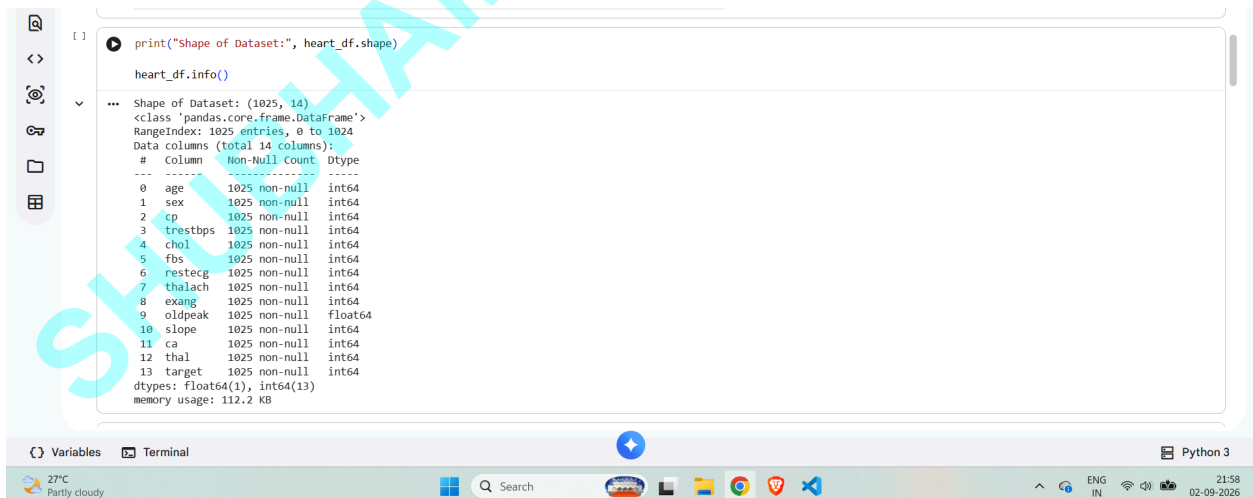
IMPORT LIBRARIES, DATA LOAD



The screenshot shows a Jupyter Notebook titled "T108Naive Bayes Classifier.ipynb". The code in the first cell imports the following libraries: numpy as np, pandas as pd, matplotlib.pyplot as plt, sklearn.model_selection for train_test_split, sklearn.naive_bayes for GaussianNB, sklearn.preprocessing for StandardScaler, and sklearn.metrics for accuracy_score, confusion_matrix, roc_curve, and auc. A print statement "Libraries Imported Successfully" is also present. The second cell loads the "heart.csv" dataset into a pandas DataFrame named heart_df and prints "Dataset Loaded Successfully". The output of the second cell shows the first five rows of the dataset as a table.

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

SHAPE OF DATASET



The screenshot shows a Jupyter Notebook with the following code in the first cell: print("Shape of Dataset:", heart_df.shape) and heart_df.info(). The output of the first line shows the shape of the dataset as (1025, 14). The output of the second line shows the information of the dataset, including the number of entries, columns, and data types.

```
Shape of Dataset: (1025, 14)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1025 non-null   int64
1   sex         1025 non-null   int64
2   cp          1025 non-null   int64
3   trestbps    1025 non-null   int64
4   chol        1025 non-null   int64
5   fbs         1025 non-null   int64
6   restecg     1025 non-null   int64
7   thalach     1025 non-null   int64
8   exang       1025 non-null   int64
9   oldpeak     1025 non-null   float64
10  slope       1025 non-null   int64
11  ca          1025 non-null   int64
12  thal        1025 non-null   int64
13  target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

FEATURES AND TARGETS

The screenshot displays a Jupyter Notebook environment with a dark-themed sidebar on the left containing icons for file management, search, and other functions. The main area shows a code cell with the following Python code:

```
[ ]  
X = heart_df.drop("target", axis=1)  
y = heart_df["target"]  
  
print("Features and Target Separated")  
print(X.head())
```

The output of the code is displayed below the cell:

```
✓ Features and Target Separated  
   age  sex    cp  trestbps   chol   fbs  restecg   thalach   exang  oldpeak  slope  \  
0    52    1    0     125    212    0         1      168       0        1.0      2  
1    53    1    0     140    203    1         0      155       1        3.1      0  
2    70    1    0     145    174    0         1      125       1        2.6      0  
3    61    1    0     148    203    0         1      161       0        0.0      2  
4    62    0    0     138    294    1         1      106       0        1.9      1  
  
   ca  thal  
0    2     3  
1    0     3  
2    0     3  
3    1     3  
4    3     2
```

Below the code cell, there is another code cell with the following code:

```
[ ] etd = StandardScaler()
```

The bottom status bar of the application shows system information: Variables, Terminal, Python 3, 27°C, Partly cloudy, Windows taskbar icons, and system clock (21:58, 02-09-2022).

DATA STANDARDIZED

```
[ ]
std = StandardScaler()
X = std.fit_transform(X)

print("Data Standardized")
print(X)

Data Standardized
[[[-0.26843658  0.66150409 -0.91575542 ...  0.99543334  1.20922066
  1.08985168]
 [-0.15815703  0.66150409 -0.91575542 ... -2.24367514 -0.73197147
  1.08985168]
 [ 1.71659547  0.66150409 -0.91575542 ... -2.24367514 -0.73197147
  1.08985168]
 ...
 [-0.81983438  0.66150409 -0.91575542 ... -0.6241209  0.23862459
 -0.52212231]
 [-0.4889957  -1.51170646 -0.91575542 ...  0.99543334 -0.73197147
 -0.52212231]
 [-0.04787747  0.66150409 -0.91575542 ... -0.6241209  0.23862459
  1.08985168]]
```

The screenshot displays a Jupyter Notebook interface with a light blue sidebar on the left containing icons for file management, search, and other functions. The main area shows a code cell with the following Python code:

```
[ ] test_size = 0.30
seed = 7

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=test_size,
    random_state=seed
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

Below the code cell, the output is displayed in a collapsed state, showing the shapes of the training and testing data:

```
... Training Data Shape: (717, 13)
Testing Data Shape: (308, 13)
```

The next code cell contains the following Python code:

```
[ ] model = GaussianNB()

print(model)
```

Below this code cell, the output is displayed in a collapsed state, showing the model object:

```
... GaussianNB()
```

The final code cell contains the following Python code:

```
[ ] model.fit(X_train, y_train)

print("Model Trained Successfully")
```

Below this code cell, the output is displayed in a collapsed state, showing the message:

```
Model Trained Successfully
```

At the bottom of the interface, there is a status bar with the following information:

- Variables: A button to view the current variables in the environment.
- Terminal: A button to open the terminal window.
- Python 3: The current Python interpreter version.
- System tray: Includes icons for network, volume, and other system utilities.
- Time and Date: 21:59, 02-09-2022.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

MODEL PREDICT CONFUSION MATRIX

```
[ ] predicted = model.predict(X_test)

print("Predicted Values")
print(predicted)

... Predicted Values
[0 1 0 0 0 1 0 1 0 0 1 1 1 0 0 0 1 1 0 1 1 1 0 0 0 1 0 0 1 0 1 0 1 0 1 1 1
 0 0 1 1 1 0 0 0 0 1 1 0 0 0 0 1 1 1 1 0 1 0 1 1 1 0 1 0 0 1 1 1 0 0 1 1 0
 0 0 1 1 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1
 0 1 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 1 1 1 0 1 0 1 0 0 1 1 1 1 1 1 0 0 1
 1 1 1 1 1 0 0 0 0 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 0 1 1
 0 1 1 1 0 1 0 0 1 1 1 1 1 0 0 1 1 0 0 1 1 1 0 1 1 0 0 0 1 0 0 1 0 1 1 0 1
 1 1 1 0 1 1 0 0 1 1 1 1 0 0 0 0 1 0 1 1 0 0 0 1 1 1 0 1 1 1 0 1 0 1 1 0 1
 1 0 0 0 1 0 0 1 1 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 0 1 1 1 1 1 1 0 1 1 0 1 0
 0 1 1 0 0 0 1 0 0 1 1 1]

[ ] cm = confusion_matrix(y_test, predicted)

print("Confusion Matrix")
print(cm)

... Confusion Matrix
[[111  30]
 [ 26 141]]

[ ] model_score = model.score(X_test, y_test)

print("Model Score:", model_score)
```

MODEL SCORE

```
[ ] model_score = model.score(X_test, y_test)

print("Model Score:", model_score)

... Model Score: 0.8181818181818182

[ ] y_predictProb = model.predict_proba(X_test)

print("Prediction Probabilities")
print(y_predictProb)

... Show hidden output

[ ] fpr, tpr, thresholds = roc_curve(y_test, y_predictProb[:,1])

roc_auc = auc(fpr, tpr)

print("ROC AUC Score:", roc_auc)

... ROC AUC Score: 0.9038094024716524

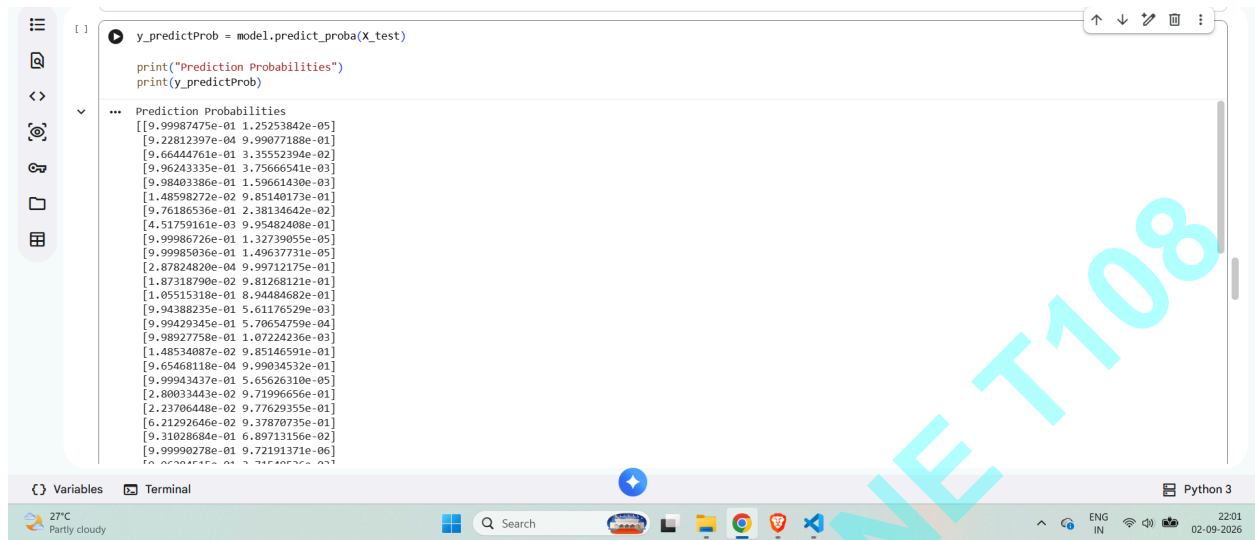
[ ] plt.figure(figsize=(7,5))

plt.plot(fpr, tpr,
         color='darkorange',
         linewidth=2,
         label='ROC Curve (AUC = %0.2f)' % roc_auc)
```

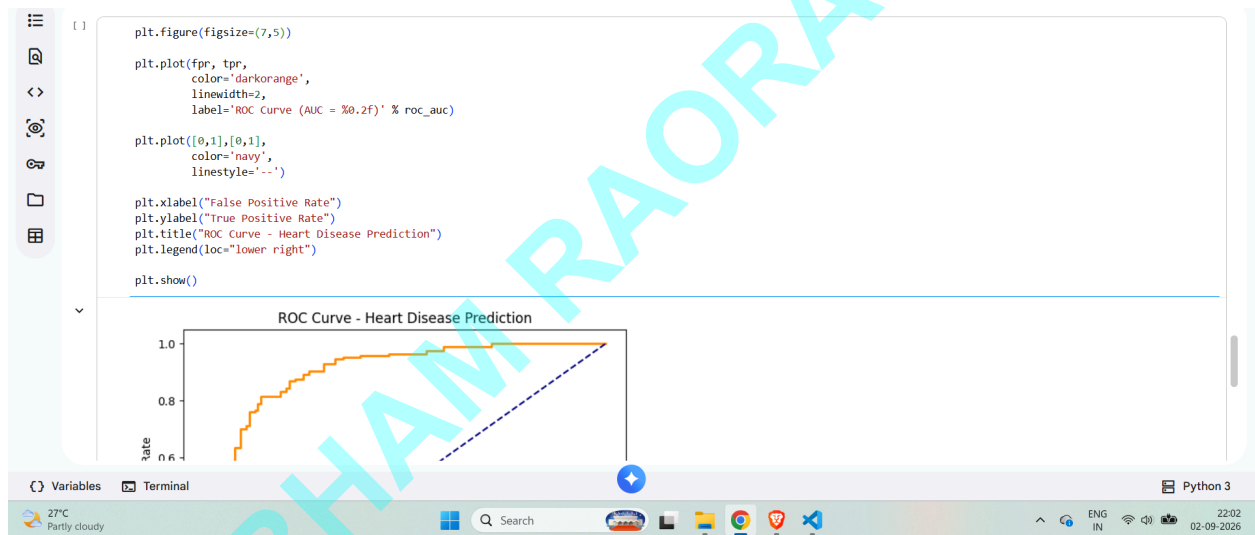
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

PREDICTION PROBABILITIES



ROC CURVE



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

